LangChain: Guia de Arquitetura e Uso

Framework para desenvolvimento de aplicações integradas com Modelos de Linguagem (LLMs)

PYTHON • LCEL • LOCAL AI

1. O que é e Para que Serve

Conceito Base

Um modelo de linguagem (como Llama, GPT ou Gemini) isolado atua apenas com base no próprio treinamento estático. O **LangChain** funciona como a camada de orquestração que conecta esses modelos a fontes de dados privadas, APIs e ferramentas externas, padronizando o desenvolvimento em quatro frentes principais:

1. Padronização
Interface única para trocar entre OpenAI, Gemini, Anthropic ou Ollama alterando apenas uma linha.

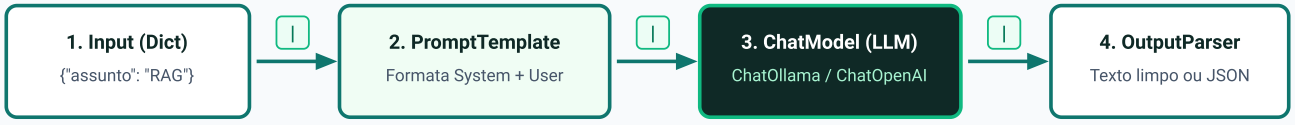
2. RAG (Dados Próprios)
Leitura de PDFs, divisão em blocos (chunks), geração de embeddings e busca em bancos vetoriais.

3. LCEL (Chains)
Composição declarativa de fluxos usando o operador pipe (`|`) para encadear etapas.

4. Agentes e Tools
Permite que a IA acione funções Python, consulte bancos SQL ou consuma APIs externas sob demanda.

2. Fluxo de Execução LCEL (LangChain Expression Language)

Pipeline Declarativo



*Sintaxe declarativa em Python: **chain = prompt | llm | parser** — Suporta execução síncrona (.invoke), lote (.batch) e tempo real (.stream).*

3. Arquitetura Modular do Ecossistema

Pacotes Independentes

Pacote / Módulo	Responsabilidade Principal	Quando Instalar
`langchain-core`	Abstrações base (Prompts, Parsers, Mensagens) e a linguagem LCEL.	Obrigatório em todo projeto.
`langchain-ollama`	Conector dedicado para rodar modelos locais via Ollama (`ChatOllama`).	Uso de IA local sem custo de API.
`langchain-openai` / `google`	Integrações oficiais isoladas por provedor em nuvem.	Apenas o provedor que for utilizar.
`langchain-community`	Carregadores de documentos (PDF, Web, SQL) e integrações de terceiros.	Projetos com RAG e leitura de arquivos.
`langgraph` / `langsmith`	**LangGraph:** Agentes com ciclos e memória. **LangSmith:** Observabilidade e debug.	Agentes autônomos e produção.

4. Implementação Prática com Ollama e Guia de Decisão

Código Pronto

```
# Instalação no ambiente virtual: uv add langchain-ollama
langchain-core
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

Quando usar LangChain?

- **Use quando:** Criar sistemas de RAG (busca em PDFs/bancos), precisar alternar entre modelos locais e nuvem, validar saídas complexas ou construir agentes